

Labo 1 : Liste des sujets

Prof. Yassin Rekik

1. Description

Vous trouverez ci-dessous une liste de quatre applications à prototyper. Pour chaque application, le style d'architecture à utiliser est imposé. Pour chaque application on vous suggère un outil qui peut vous être utile pour l'implémentation. Vous devez modéliser l'architecture de votre application à l'aide des ADLs vus au cours, à savoir : UML, SysML et DFD puis faire une implémentation simple (prototype) de cette application.

Pour chaque application la modélisation devra contenir au minimum :

- un diagramme de cas d'utilisation
- un diagramme de DFD level 1 ou plus
- un diagramme de composant et déploiement
- un diagramme de requirement
- un diagramme de classe ou un block definition diagramme (BDD)

Il se peut que, pour certaines applications, certains diagrammes soient très simples. Si nécessaire d'autres diagrammes peuvent être utilisés (en plus, pas à la place). La modélisation devra faire clairement apparaître le type d'architecture utilisé (imposé).

Le rendu du laboratoire consistera en :

1. Un rapport de modélisation dans lequel vous décrivez de manière textuelle l'application que vous avez implémentée ainsi que la modélisation formelle au moyen des diagrammes avec les explications nécessaires à la compréhension de votre modèle.
2. Les codes source et exécutable de votre application.
3. Une présentation orale de votre travail suivi d'une courte démonstration.

Ce travail s'effectuera en groupe. La qualité des interfaces n'est pas du tout significative. Même des interfaces commandes seront acceptées.

2. Liste des sujets

2.1: **Sujet 1 : Architecture « Layer based »**

Le but est de développer une application interactive de visualisation du chemin le plus court sur un terrain. Un terrain NxN de cases est affiché. L'utilisateur peut désigner la case de départ, la case d'arrivée et les coûts des différentes cases du terrain. Et à chaque modification sur le terrain, le chemin le plus court est recalculé et affiché. L'application est Standalone, mais elle doit être conçue en 3 couches : une couche de visualisation et d'interaction avec l'utilisateur, une couche de gestion de l'état du terrain, une couche de calcul de chemin le plus court.

Outil suggéré : Java Swing et Implémentation Java de l'algorithme A*

2.2. **Sujet 2 : Architecture “Object based”**

On veut développer une application de « chat » point à point au sein d'un groupe d'utilisateurs. À tout moment un membre peut demander un « chat » avec un autre membre, même si celui-ci a déjà d'autres « chat » en cours. Un participant peut accepter ou refuser une demande de « chat », comme il peut arrêter le « chat » quand il en a envie. Au bout d'un timeout, un chat non accepté est considéré comme refusé.

Outil suggéré : Java RMI.

2.3. **Sujet 3 : Architecture “Event based”**

Le but est de simuler une application de type “publish/subscribe” permettant de s'abonner à différent type des “news” (sport, économie, politique, chiens/chats, etc..). Il y a des éditeurs (publishers) et des abonnés (subscriber). Chaque fois qu'un éditeur rejoint ou quitte le système il l'annonce de manière à ce que chaque abonné connaisse les éditeurs présents dans le système. Les abonnés peuvent s'abonner ou se désabonner à un ou plusieurs éditeurs. Chaque fois qu'un éditeur publie une nouvelle, tous les abonnés à cet éditeur reçoivent la nouvelle.

Outil suggéré : Java listener

2.4. **Sujet 4 : Architecture “Data centered”**

Il s'agit de développer une version simplifiée du jeu de Roulette. L'ordinateur joue le rôle de la banque. Les joueurs jouent tous sur un tapis partagé. Les joueurs disposent d'un intervalle de temps pour les paris. Une case prise par un joueur ne peut pas recevoir d'autres paris. Par contre, chaque joueur peut changer ses paris tant que le temps le permet. Une fois les paris clos, l'ordinateur fait un tirage Random et distribue les gains selon les paris effectués.

Outil suggéré : Web socket